

Java

1. Sequential Theory: The Evolution of Java

- **Phase 1: The Green Project (1991)** Initiated by **James Gosling**, Mike Sheridan, and Patrick Naughton at Sun Microsystems. The goal was to develop a language for digital consumer electronics (like set-top boxes).
- **Phase 2: The "Oak" Era (1992)** The language was initially named **Oak** (after an oak tree outside Gosling's office). It was designed to be platform-independent and robust.
- **Phase 3: The Pivot to Web (1994)** The team realized Oak was perfect for the burgeoning World Wide Web. They created **WebRunner** (later HotJava), a browser that could run interactive code.
- **Phase 4: The Birth of Java (1995)** Oak was renamed **Java** (after coffee) because "Oak" was already trademarked. It was officially announced at the SunWorld conference.
- **Phase 5: JDK 1.0 & WORA (1996)** The first stable version was released with the promise: **"Write Once, Run Anywhere."** This meant code compiled on one system could run on any other with a JVM.
- **Phase 6: Enterprise and Mobile (1998-2006)** Java split into three editions: **J2SE** (Standard), **J2EE** (Enterprise), and **J2ME** (Mobile), dominating the server-side and early mobile phone markets.
- **Phase 7: Oracle Acquisition (2010)** **Oracle Corporation** acquired Sun Microsystems. Java development continued under Oracle's stewardship.
- **Phase 8: Modern Era (2014-Present)** **Java 8** introduced functional programming. Since **Java 9**, Oracle moved to a **6-month rapid release cycle** to keep the language modern and competitive.

2. Real-World Project Use: Android OS

In the mid-2000s, **Google** chose Java as the primary language for **Android**. Because of Java's history of portability and its vast library ecosystem, Google was able to build a mobile platform that could run on thousands of different hardware configurations, eventually becoming the most used OS in the world.

3. Short Notes

- **Father of Java:** James Gosling.
- **Original Name:** Oak.
- **Core Philosophy:** Security, Portability, and Object-Orientation.
- **Key Slogan:** WORA (Write Once, Run Anywhere).
- **Current Owner:** Oracle Corporation.

1. Data Types

Data types specify the size and type of values that can be stored. They are divided into **Primitive** and **Non-Primitive**.

Primitive Data Types

Type	Size	Scanner Method	BufferedReader + Parsing
byte	1 byte	<code>.nextByte()</code>	<code>Byte.parseByte(reader.readLine())</code>
short	2 bytes	<code>.nextShort()</code>	<code>Short.parseShort(reader.readLine())</code>
int	4 bytes	<code>.nextInt()</code>	<code>Integer.parseInt(reader.readLine())</code>
long	8 bytes	<code>.nextLong()</code>	<code>Long.parseLong(reader.readLine())</code>
float	4 bytes	<code>.nextFloat()</code>	<code>Float.parseFloat(reader.readLine())</code>
double	8 bytes	<code>.nextDouble()</code>	<code>Double.parseDouble(reader.readLine())</code>
boolean	1 bit	<code>.nextBoolean()</code>	<code>Boolean.parseBoolean(reader.readLine())</code>

char	2 bytes	<code>.next().charAt(0)</code>	<code>(char)reader.read()</code>
------	---------	--------------------------------	----------------------------------

Real-World Project Use: In a **FinTech application**, double is used for account balances, while boolean tracks if a transaction is "verified."

2. Variables

A variable is a container that holds data during Java program execution.

- **Local Variables:** Declared inside a method; cannot be accessed outside.
- **Instance Variables:** Declared inside a class but outside methods; specific to an object.
- **Static Variables:** Declared with the static keyword; shared across all instances of a class.

Short Notes:

- Names must start with a letter, \$, or _.
 - Java is **case-sensitive** (myVar vs myvar).
 - Use **camelCase** for naming (e.g., userAge).
-

3. Operators

Operators are symbols used to perform operations on variables and values.

Category	Operators	Description
Arithmetic	<code>+, -, *, /, %</code>	Basic math and modulo (remainder).
Relational	<code>==, !=, >, <, >=, <=</code>	Comparison (returns boolean).
Logical	<code>&&, ` </code>	
Assignment	<code>=, +=, -=, *=, /=</code>	Assigning and updating values.

Unary	++, --	Increment and decrement by 1.
-------	--------	-------------------------------

4. Input and Output (I/O)

In Java, we use the Scanner class for input and System.out for output.

- **Output:**
 - print(): Stays on the same line.
 - println(): Moves to a new line after printing.
- **Input:**
 - next(): Reads a single word.
 - nextLine(): Reads a full line of text.
 - nextInt(): Reads an integer.

The "Coffee Tracker" Program

This program calculates the total cost of coffee based on user input.

Java

```
import java.util.Scanner;

public class CoffeeTracker {
    public static void main(String[] args) {
        // Create Scanner object for input
        Scanner input = new Scanner(System.err);

        // Variables and Data Types
        String coffeeName = "Espresso";
        double pricePerCup = 3.50;

        System.out.print("How many cups of " + coffeeName + " did you drink? ");
        int quantity = input.nextInt(); // Input: Integer

        // Operator: Multiplication
        double totalBill = pricePerCup * quantity;
```

```
// Output
System.out.println("--- Receipt ---");
System.out.println("Item: " + coffeeName);
System.out.println("Total Cost: $" + totalBill);

    input.close();
}
}
```